A Programming Paradigm for Spatiotemporal Composability

Yifan Shi[1,2], Wei Zhang[1], Tianyi Cui[2]

[1]**Peking University** [2]**DeepSeek-AI**

Abstract

Modern software—from plugin systems to self-evolving agent harnesses—increasingly requires *dynamic composition*, yet its formal foundations remain underdeveloped. We identify two orthogonal dimensions of the problem: *temporal composability*, the ability to completely revert a component's side effects upon removal, and *spatial composability*, the ability to declare and reactively manage inter-component dependencies. We address the two dimensions by lifting classical effect and coeffect concepts to runtime mechanisms. In particular, we formalize *revertible effects*, in which every context transformation carries an inverse that the runtime holds, establishing temporal composability local to one component. We formalize *reactive coeffects*, in which every context change is classified against a component's coeffect specification to drive its activation and deactivation, establishing spatial composability local to one component. We then unify the effect context and the coeffect context into a single context type and mediate every effect and coeffect through it, yielding a discipline we call the *context paradigm*; the mediation induces an observational equivalence up to which the effects of distinct components interleave without disturbing one another. Combining these mechanisms into the notion of a *component*, we give a calculus of dynamic composition whose metatheory carries spatiotemporal composability from a single component to a whole system of interleaved components. We implement these ideas in *Cordis*, a meta-framework of spatiotemporal composability that provides a core library with effect tracking and coeffect resolution, as well as a declarative component loader with configuration reconciliation and hot module replacement.

Contents

1. Introduction

Composition—assembling complex systems from simpler parts—is a foundational principle of software engineering [1]. Traditionally, composition is *static*: function calls, module imports, and class inheritance are resolved at compile time and remain fixed throughout execution. However, modern software increasingly demands *dynamic* composition, where components are loaded, unloaded, and reconfigured at runtime. Plugin architectures [2] and self-evolving agent harnesses both require systems that can safely add and remove functionality on the fly, yet current practice defers to coarse-grained mechanisms [3] that reconfigure only by restarting, discarding runtime state. Despite the growing practical importance of dynamic composition, its theoretical foundations remain underdeveloped, compared to the rich formal frameworks available for static composition.

1.1. Dimensions of Composability

To characterize the requirements of dynamic composition, we identify two orthogonal dimensions beyond the well-studied algebraic aspects of composition:

- **Temporal composability** addresses the *time* dimension: upon removal of a component, the modifications the component made to the shared environment must be completely and safely reversed. This requires tracking every resource allocation, event registration, and state mutation the component performs, and guaranteeing their orderly reclamation upon removal.
- **Spatial composability** addresses the *space* dimension: components must be able to declare, discover, and resolve their dependencies on one another in a structured and verifiable manner. This requires managing dependency topology and coordinating component lifecycles in response to dependency changes.

In the static setting, temporal composability reduces to lexical scoping (e.g., RAII [4], bracket patterns [5]), and spatial composability reduces to module import resolution [6]. In the dynamic setting, where components arrive and depart at runtime, both dimensions become significantly harder: temporal composability must handle long-lived, stateful effects whose scope is not lexically bounded; and spatial composability must handle dependencies that appear, disappear, or change identity during execution.

1.2. Motivating Examples

1.2.1. Plugin Systems

Plugin systems are a canonical instance of dynamic composition. We use Visual Studio Code (VSCode), one of the most widely-used extensible IDEs, as a representative example.

Temporal limitation. VSCode runs all extensions in a shared process called the extension host. Although extensions can be installed dynamically, this host provides no mechanism to unload an individual extension's code at runtime. Once an extension's `activate` function has executed, disabling or uninstalling it requires restarting the entire host, affecting all loaded extensions. Purely declarative extensions such as themes, keybindings, and snippets carry no

code and can be removed freely. Among the top 100 extensions by install count, however, 87 contain executable code[1] and will therefore require such a restart upon removal. Although VSCode provides a `deactivate` hook, it serves only as a graceful shutdown callback during the host process' termination, and thus does not enable live removal. Moreover, the hook separates effect disposal from effect creation (in `activate`), violating locality of concern and making complete cleanup difficult to verify.

Spatial limitation. VSCode does provide `extensionDependencies` for declaring dependencies between extensions, but it sees little use: among the top 100 extensions by install count, only 7 declare `extensionDependencies` on non-built-in extensions.[1] This scarcity reflects the shape of the extension API, which exposes fixed, surface-level extension points such as commands, views, and language features. Extensions contribute to the host through these points rather than depending on one another, so inter-extension dependencies rarely arise. Moreover, VSCode's mechanism for inter-extension interaction provides no structural contract: it exposes an extension's functionality to others through `vscode.extensions.getExtension(...).exports`, but the returned value is untyped (`any` by default), so the dependent cannot rely on a checked interface. In short, VSCode steers extensions toward a fixed set of host-provided extension points, and offers no safe, structured way for them to depend on one another.

These two limitations are not unique to VSCode; they recur across plugin systems generally [2, 7], differing only in degree.

1.2.2. Self-Evolving Agent Harnesses

Modern AI agents rely on runtime agent harnesses [8–10]. These systems may compose diverse tool suites [11] and execution environments, govern permissions and sandboxing, maintain session state and persistence, provide context management and memory systems [12], orchestrate subagents and multi-agent workflows [13], and expose interfaces to users and automation. A future harness may generate and deploy modifications to its own components while continuously serving requests. Model-synthesized reusable tools provide a narrower precursor to component-level self-modification [14]. Each such modification is itself an instance of dynamic composition.

Because these modifications occur continuously and with limited or no human oversight, dynamic composability becomes indispensable. Without temporal composability, each self-modification forces a full restart that discards all process-local accumulated state; at such frequency the cumulative unavailability becomes substantial, and in-flight tasks are disrupted repeatedly; even worse, a faulty self-modification can disable the very process needed to recover. Without spatial composability, each module must itself detect and adapt to changes in the modules it depends on as they appear, disappear, or change identity, and can do so only by ad hoc means; even worse, a naive code-replacement strategy may silently break dependents or introduce circular dependencies that surface only at reload time.

1.2.3. The Coarse-Grained Workaround

One reason dynamic composability has received limited formal attention is that operating systems and container orchestrators already provide a coarse-grained substitute. Operating systems yield temporal composability at the granularity of a process; container orchestrators

[1] Data retrieved from the Visual Studio Code Marketplace on June 9, 2026

[3] yield spatial composability at the granularity of a service. In practice, most software tolerates the lack of fine-grained composability by deferring to these coarse-grained mechanisms: a misbehaving module is handled by restarting the process, and a service dependency is managed by the container orchestrator.

However, this workaround imposes substantial costs. Temporally, each restart discards all process-local accumulated state (e.g., caches, connections, partial computations), and rebuilding it takes seconds to minutes [15]; maintaining availability in the interim requires redundant replicas, incurring resource overhead to compensate for the inability to recover a single component. Spatially, container-level orchestration cannot express dependencies between components sharing an address space, and introduces network overhead for interactions that could be local function calls. Both mechanisms operate at the boundary of processes and containers, yet modern systems increasingly compose at a finer level. This granularity mismatch demands a compositional abstraction that manages effects and dependencies at the same level as the components themselves.

1.3. Contributions

The two dimensions of dynamic composability concern, respectively, how computations *modify* and how they *depend on* their environment. These two directions are what *effect systems* [16, 17] and *coeffect systems* [18, 19] formalize: effects provide the formal vocabulary for reasoning about environmental modifications, and coeffects for reasoning about environmental requirements. However, existing formulations restrict reasoning to compile-time analysis over lexically fixed scopes, and do not extend to dynamic scenarios where components arrive and depart at runtime. By lifting effects to a revertible runtime model and coeffects to a reactive dependency resolution mechanism, we obtain a unified formal foundation for dynamic composability, one that is language-agnostic and applicable to any software architecture requiring dynamic composition. We make the following contributions:

1. We formalize **revertible effects** (Section 3.1): every context transformation carries an explicit inverse that the runtime holds, and both tracking and recovery preserve composition, so the context is recovered upon component removal. This establishes local temporal composability.
2. We formalize **reactive coeffects** (Section 3.2): a component declares the coeffects it requires as a specification, and each change of the context is classified against that specification as activating, deactivating, or neutral, driving the component's activation and deactivation. This establishes local spatial composability.
3. We introduce the **context paradigm** (Section 3.3): the effect context and the coeffect context are unified into a single context type, every effect and coeffect is mediated through it, and the mediation induces an observational equivalence up to which the effects of distinct components attain independence.
4. We develop a calculus of dynamic composition (Section 4), which combines the two mechanisms into the notion of a **component** and gives them an operational semantics. The metatheory then carries spatiotemporal composability from a single component to a whole system of interleaved components.
5. We implement these ideas in **Cordis** (Section 5), a meta-framework of spatiotemporal composability that provides a core library realizing the formal model with effect tracking and coeffect resolution, as well as a declarative component loader with configuration reconciliation and hot module replacement.

2. Preliminaries

This section provides a concise overview of effect and coeffect systems—the two theoretical pillars underlying our work. We assume familiarity with basic type theory and category theory; the goal here is to fix notation and introduce the key abstractions that Section 3 will operationalize as runtime mechanisms.

2.1. Effects

In the simply typed lambda calculus (STLC) [20, 21], a typing judgment $\Gamma \vdash t : T$ states that term t has type T under context Γ. An *effect system* refines the type to describe what side effects a computation may produce, yielding judgments of the form

$$\Gamma \vdash t : T_{\text{effect}} \tag{1}$$

Here, the result type is annotated with an element of an effect algebra that describes which side effects the computation may produce, enabling compositional reasoning about stateful computations. This approach originates with Lucassen and Gifford [22], who introduced a kinded type system distinguishing types, effects, and regions to discover scheduling constraints in parallel programs.

Monadic effects. Moggi [16] first modeled computational effects categorically via monads; Wadler [23] popularized the approach in Haskell. A monad (T, η, μ) on a category $\mathcal{C}$ encapsulates an effectful computation as a value of type $T(A)$, with $\eta : A \to T(A)$ lifting pure values and $\mu : T(T(A)) \to T(A)$ sequencing nested computations. Classic instances include the Maybe monad (for partiality), State monad (for mutable state), and IO monad (for external interaction).

Algebraic effects. Plotkin and Power [17, 24] showed that algebraic operations determine monads, establishing a framework in which effect interfaces are decoupled from their implementations. An effect signature Σ declares a set of operations (e.g., $\text{get} : () \to S$, $\text{put} : S \to ()$ for state); programs invoke operations freely without committing to a particular interpretation. Plotkin and Pretnar [25] subsequently introduced *effect handlers*, which interpret operations by providing continuation semantics:

$$\text{handle } e \text{ with } \{ \text{ op}(v, \kappa) \mapsto \ldots \} \tag{2}$$

The handler receives the operation argument v and the delimited continuation κ, which it may invoke zero, one, or multiple times, enabling exceptions, coroutines, and non-determinism within a uniform framework [26]. Languages such as Koka [27, 28], Eff [29], and OCaml 5 [30] have adopted algebraic effects with varying design trade-offs.

2.2. Coeffects

Dually to effects, a *coeffect system* [18, 31] enriches the context rather than the type, yielding judgments of the form

$$\Gamma_{\text{coeffect}} \vdash t : T \tag{3}$$

Here, the context is annotated with an element of a coeffect algebra describing what the computation requires from its environment, such as resources to access, permissions to hold,

or services to depend on. While effects model a program's impact on the world, coeffects model the world's constraints on the program.

Comonadic coeffects. The idea of using comonads to structure context-dependent computation was first developed by Uustalu and Vene [32], who proposed symmetric (semi)monoidal comonads as the dual of Moggi's monadic framework for effects, capturing notions such as dataflow and attribute evaluation. Petricek et al. [18] built on this foundation to propose coeffects as a unified static analysis of context-dependence. A comonad (D, ε, δ) captures context-dependent computation: $\varepsilon : D(A) \to A$ extracts the current value from a context, and $\delta : D(A) \to D(D(A))$ duplicates context for nested access. The Environment comonad $D(X) = E \times X$ models dependence on a fixed environment E; the Stream comonad $D(X) = \mathbb{N} \to X$ models dependence on temporal data.

Graded coeffects. For finer-grained tracking, *graded* coeffect systems use a pre-ordered semiring $\mathcal{S} = (S, \leq, +, \times, 0, 1)$ as the coeffect algebra [33], a discipline later unified with graded effects by Gaboardi et al. [19]. Elements of S annotate each variable binding to quantify its usage: 0 for unused, 1 for linear use, n for bounded use, ∞ for unrestricted use. The semiring operations compose coeffects sequentially $(\times)$ and in parallel $(+)$, enabling precise resource tracking, sensitivity analysis [34], and information-flow control [35, 36] within a unified algebraic framework [37].

2.3. Relationship to Dynamic Composability

Effect and coeffect systems organize reasoning about computation along two complementary directions: effects describe how a computation *modifies* its environment, whereas coeffects describe how it *depends on* its environment. These two directions correspond to the two dimensions of dynamic composability identified in Section 1:

- **Temporal composability** demands that a component's modifications to the shared environment be revertible upon unloading. The relevant effects are the stateful ones, which durably transform that environment; undoing such a transformation requires it to admit an inverse.
- **Spatial composability** demands that inter-component dependencies be declared and managed reactively. Such dependencies are the very thing coeffects capture, and managing them amounts to resolving each against what the environment supplies.

However, classical effect and coeffect systems are static instruments: effects are tracked within lexically fixed scopes and discharged by compile-time handlers; coeffect annotations are verified against contexts determined before execution. Dynamic composition, by contrast, requires these guarantees to hold for components that arrive and depart at runtime, against contexts that evolve continuously. No fixed lexical scope can delimit a plugin loaded after deployment; no compile-time context can anticipate dependencies that emerge from runtime configuration.

This motivates a shift in perspective: rather than extending static type systems with more annotations, we reify the conceptual structures of effects and coeffects so that a runtime can operate on them directly, establishing dynamically the guarantees these systems provide statically.

3. Revertible Effects and Reactive Coeffects

This section lifts the concepts of effects and coeffects introduced in Section 2 to runtime mechanisms, constructing a theory of dynamic composition. The central idea is to turn the *typing contexts* carrying effects and coeffects into *context types*, runtime-operable types that reify the context as a first-class entity. Section 3.1 models an effect as a context transformation paired with an inverse that the runtime holds, establishing temporal composability local to one component; Section 3.2 models a coeffect as a declared dependency against which every context change is classified, establishing its spatial counterpart. Each local guarantee stops where other components enter. Toward the global form of both, Section 3.3 unifies the two contexts into one and introduces the *context paradigm*: every effect and coeffect is mediated through the unified context, and the mediation induces the observational equivalence up to which every later equality is read. Section 3.4 then establishes effect independence and coeffect commutativity, under which the effects of distinct components interleave without disturbing one another.

3.1. Revertible Effects

Temporal composability is the ability to load and unload components at runtime such that, upon unloading, the shared environment is recovered to its pre-composition state. This requires that every modification a component makes to the environment be both trackable and recoverable. We therefore model an effect as a function of type $\Gamma \to \Gamma \times (\Gamma \to \Gamma)$: applied to the current context, it yields the modified context together with an explicit inverse. Supplying that inverse is what lets the effect be reverted, and returning it to the runtime is what makes the effect trackable. We call such effects *revertible*: by composing these inverses during execution, local temporal composability becomes a structural guarantee.

3.1.1. Effect Context

Given any impure function $f : X \rightsquigarrow Y$, we transform it into a pure form $f : \Gamma \times X \to \Gamma \times Y$, where Γ is the context type. On this pure form, all possible side effects can be represented as transformations on Γ: for any fixed input $x : X$, the induced map $\gamma \mapsto \mathrm{pr}_1(f(\gamma, x)) : \Gamma \to \Gamma$ captures the side effect of f independently of the return value. Effects on Γ therefore live in the monoid of transformations $\Gamma \to \Gamma$ under composition $\circ$, where each monoid axiom has a direct reading as a property of effects:

- *Closure*: the sequential composition of two effects is again an effect;
- *Associativity*: a composite effect is independent of how it is bracketed;
- *Identity*: id_Γ, the identity function on Γ, acts as the unit of composition.

To model effects that can be undone, we pair each transformation f with another transformation g that undoes f, and call g a *left inverse* of f, abbreviated to *inverse* throughout the paper. Undoing is one-sided: what an inverse is held to is $g \circ f$ and never $f \circ g$. Pairs of transformations carry a multiplication of their own:

Definition 1. Define the *twisted composition* of pairs of context transformations by

$$(f_1, g_1) \circ (f_2, g_2) := (f_1 \circ f_2, g_2 \circ g_1) \tag{4}$$

As for $\circ$ itself, the left operand acts after the right, and the inverses accumulate in the opposite order. It makes $(\Gamma \to \Gamma) \times (\Gamma \to \Gamma)$ a monoid with unit $(\mathrm{id}_\Gamma, \mathrm{id}_\Gamma)$, the product of the monoid of transformations with its opposite, which we call the twisted composition monoid $\mathfrak{T}_\Gamma$ over Γ.

To track effects within the context itself, we introduce the following definition:

Definition 2. Given a context Γ, define its *effect context* as:

$$\partial\Gamma := \Gamma \times (\Gamma \to \Gamma) \tag{5}$$

It can be understood as a pair (γ, φ), where:

- $\gamma : \Gamma$ is the current context state;
- $\varphi : \Gamma \to \Gamma$ is the *accumulator*, the composite of the inverses of the effects performed so far, and the function that recovers the context to its initial state.

In particular, the initial effect context can be represented as $(\gamma_0, \mathrm{id}_\Gamma)$.

We also write $\partial^2\Gamma$ for $\partial(\partial\Gamma) = \partial\Gamma \times (\partial\Gamma \to \partial\Gamma)$; iterating ∂ this way yields the tower $\Gamma, \partial\Gamma, \partial^2\Gamma, \cdots$.

Given the presence of the accumulator φ, all effects performed on $\partial\Gamma$ can be tracked and the context can be recovered. We now give the concrete constructions for tracking and recovery.

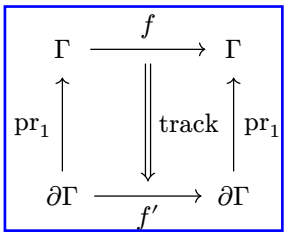
Definition 3. Define the transformation track_Γ on pairs of context functions:

$$\begin{array}{lcccccc} \mathrm{track}_\Gamma & : & (\Gamma \to \Gamma) \times (\Gamma \to \Gamma) & \to & \partial\Gamma & \to & \partial\Gamma \\ \mathrm{track}_\Gamma & = & (f, g) & \mapsto & (\gamma, \varphi) & \mapsto & (f(\gamma), \varphi \circ g) \end{array} \tag{6}$$

This transformation converts a forward function f together with a candidate inverse g into a transformation of the effect context $\partial\Gamma$. Applying $\mathrm{track}_\Gamma(f, g)$ to a state (γ, φ) transforms γ by f and composes the inverse g onto φ, thereby tracking the effect of f in the context.

Theorem 4. For every $(f, g) \in (\Gamma \to \Gamma) \times (\Gamma \to \Gamma)$, write $f' := \mathrm{track}_\Gamma(f, g)$; then the following diagram commutes, that is,

$$\mathrm{pr}_1 \circ f' = f \circ \mathrm{pr}_1 \tag{7}$$



Proof. For all $(\gamma, \varphi) \in \partial\Gamma$:

$$\begin{aligned} (\mathrm{pr}_1 \circ \mathrm{track}_\Gamma(f, g))(\gamma, \varphi) &= \mathrm{pr}_1(f(\gamma), \varphi \circ g) \\ &= f(\gamma) \\ &= (f \circ \mathrm{pr}_1)(\gamma, \varphi) \end{aligned}$$

□

Theorem 4 ensures that tracking leaves the forward behavior untouched: on the context state, $\mathrm{track}_\Gamma(f, g)$ acts as f does, whatever candidate inverse it carries.

Theorem 5. track_Γ is a monoid homomorphism from $\mathfrak{T}_\Gamma$ into $\partial\Gamma \to \partial\Gamma$. That is,

1. $\mathrm{track}_\Gamma(\mathrm{id}_\Gamma, \mathrm{id}_\Gamma) = \mathrm{id}_{\partial\Gamma}$;
2. for all $(f_1, g_1), (f_2, g_2) \in \mathfrak{T}_\Gamma$,

$$\mathrm{track}_\Gamma((f_1, g_1) \circ (f_2, g_2)) = \mathrm{track}_\Gamma(f_1, g_1) \circ \mathrm{track}_\Gamma(f_2, g_2) \tag{8}$$

Proof.

1. The unit is carried to the unit, since $\mathrm{track}_\Gamma(\mathrm{id}_\Gamma, \mathrm{id}_\Gamma)(\gamma, \varphi) = (\gamma, \varphi \circ \mathrm{id}_\Gamma) = (\gamma, \varphi)$.
2. For the multiplication, take any $(\gamma, \varphi) \in \partial\Gamma$:

$$\begin{aligned}(\mathrm{track}_\Gamma(f_1, g_1) \circ \mathrm{track}_\Gamma(f_2, g_2))(\gamma, \varphi) &= \mathrm{track}_\Gamma(f_1, g_1)(f_2(\gamma), \varphi \circ g_2) \\ &= (f_1(f_2(\gamma)), \varphi \circ g_2 \circ g_1) \\ &= \mathrm{track}_\Gamma(f_1 \circ f_2, g_2 \circ g_1)(\gamma, \varphi)\end{aligned}$$

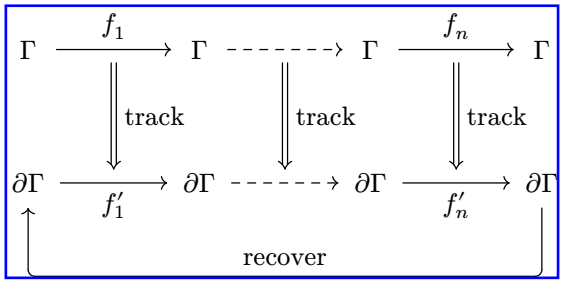
□

Theorem 5 ensures that tracking one pair at a time agrees with tracking their twisted composite at once, so a sequence of tracked effects can be reasoned about as a single tracked effect.

Definition 6. Define the transformation $\mathrm{recover}_\Gamma$ on $\partial\Gamma$:

$$\begin{array}{lcccc} \mathrm{recover}_\Gamma & : & \partial\Gamma & \to & \partial\Gamma \\ \mathrm{recover}_\Gamma & = & (\gamma, \varphi) & \mapsto & (\varphi(\gamma), \mathrm{id}_\Gamma) \end{array} \tag{9}$$

This transformation applies the recovery function φ to the current state γ and resets φ to the identity. The following diagram illustrates how recover recovers the context to its initial state after a sequence of effects $f_i' := \mathrm{track}_\Gamma(f_i, g_i), i = 1, \cdots, n$, has been applied to $\partial\Gamma$:



The diagram shows that the tracked effects followed by recover carry the initial effect context back to itself. Each tracking step in fact preserves the result of recovery itself, from whatever state it is taken:

Theorem 7. For every $(\gamma, \varphi) \in \partial\Gamma$ and every pair (f, g) with $g(f(\gamma)) = \gamma$,

$$\mathrm{recover}_\Gamma(\mathrm{track}_\Gamma(f, g)(\gamma, \varphi)) = \mathrm{recover}_\Gamma(\gamma, \varphi) \tag{10}$$

Proof.

$$\begin{aligned}\mathrm{recover}_\Gamma(\mathrm{track}_\Gamma(f, g)(\gamma, \varphi)) &= \mathrm{recover}_\Gamma(f(\gamma), \varphi \circ g) \\ &= (\varphi(g(f(\gamma))), \mathrm{id}_\Gamma) \\ &= (\varphi(\gamma), \mathrm{id}_\Gamma) = \mathrm{recover}_\Gamma(\gamma, \varphi)\end{aligned}$$

□

Theorem 7 ensures that a tracked effect whose inverse reverts it does not move the result of recovery: recovering after the step returns what recovering before it would have. Recovery reads a state through the quantity $\varphi(\gamma)$ alone, so the guarantee amounts to preserving $\varphi(\gamma)$; we refer to $\varphi(\gamma) = \gamma_0$ as the *soundness invariant* of a state in $\partial\Gamma$. In particular, starting from the initial effect context $(\gamma_0, \mathrm{id}_\Gamma)$, every state reached by tracked effects whose inverses revert them satisfies the invariant, and recovery carries each such state back to $(\gamma_0, \mathrm{id}_\Gamma)$.

The preservation along a sequence follows from Theorem 5 in one application. Let $(f_1, g_1), \cdots, (f_n, g_n)$ be applied in order from (γ, φ), and write $\delta_0 = \gamma$ and $\delta_i = f_i(\delta_{i-1})$ for the intermediate context states. By Theorem 5, the composite $\mathrm{track}_\Gamma(f_n, g_n) \circ \cdots \circ \mathrm{track}_\Gamma(f_1, g_1)$ is a single tracking step, track_Γ of the twisted composite $(f_n \circ \cdots \circ f_1,\ g_1 \circ \cdots \circ g_n)$. If each inverse reverts its own step, $g_i(\delta_i) = \delta_{i-1}$, then the composite inverse carries δ_n back to the start: $(g_1 \circ \cdots \circ g_n)(\delta_n) = \delta_0 = \gamma$. The twisted composite therefore meets the hypothesis of Theorem 7 at γ, and one application of the theorem gives

$$\mathrm{recover}_\Gamma((\mathrm{track}_\Gamma(f_n, g_n) \circ \cdots \circ \mathrm{track}_\Gamma(f_1, g_1))(\gamma, \varphi)) = \mathrm{recover}_\Gamma(\gamma, \varphi) \tag{11}$$

A pair with $g \circ f = \mathrm{id}_\Gamma$ meets the hypothesis at every state.

3.1.2. Effect Functions

The track/recover model of the previous section has two limitations:

1. $\mathrm{track}_\Gamma(f, g)$ fixes g before any context state is seen, so one uniform g has to meet the hypothesis of Theorem 7 at every state the effect is applied at. Reverting needs less: an inverse for the one state where f is applied, which may differ from state to state. A per-state inverse cannot be fixed in the argument position before the state is seen; it has to be returned at the point of application.
2. $\mathrm{recover}_\Gamma$ is all-or-nothing: it cannot selectively undo one effect while retaining others.

To address the two issues, we enhance the model at the input and output sides respectively:

1. On the input side, we not only transform Γ but also return an inverse function alongside it, so that the inverse is supplied where the effect is applied: $\Gamma \to \Gamma \times (\Gamma \to \Gamma)$, i.e., $\Gamma \to \partial\Gamma$;
2. On the output side, we not only transform $\partial\Gamma$ but also return an inverse function alongside it, so that one effect can be undone while the others are retained: $\partial\Gamma \to \partial\Gamma \times (\partial\Gamma \to \partial\Gamma)$, i.e., $\partial\Gamma \to \partial^2\Gamma$.

The two changes give the input and the output the same shape, i.e., a map from a context to the transformed context paired with an inverse: $\Gamma \to \partial\Gamma$ on the input side and $\partial\Gamma \to \partial^2\Gamma$ on the output side. One type family therefore covers both levels, and we define it at each context as the effect function type $\mathfrak{E}_\Gamma$, refined by a witness to $\mathfrak{E}^*_\Gamma$:

Definition 8. Define the effect function $\mathfrak{E}_\Gamma$ and *witnessed* effect function $\mathfrak{E}^*_\Gamma$ as:

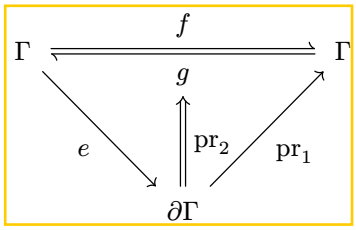
$$\begin{aligned} \mathfrak{E}_\Gamma &:= \Gamma \to \Gamma \times (\Gamma \to \Gamma) \\ \mathfrak{E}^*_\Gamma &:= (e : \Gamma \to \Gamma \times (\Gamma \to \Gamma)) \\ &\quad \times ((\gamma : \Gamma) \to (\delta : \Gamma) \to (g : \Gamma \to \Gamma) \to ((\delta, g) = e(\gamma) \to g(\delta) = \gamma)) \end{aligned} \tag{12}$$

where $e(\gamma)$ yields a pair (δ, g) representing:

- $\delta : \Gamma$ is the new context;
- $g : \Gamma \to \Gamma$ is the inverse function of the current effect.

The witness holds each returned inverse to one equation $g(\delta) = \gamma$: the inverse is required to revert the effect only at the state where it was applied. An element of $\mathfrak{E}^*_\Gamma$ may therefore choose a different inverse at every state. A single g with $g \circ f = \mathrm{id}_\Gamma$ meets the equation at every state at once, so the assignment $(f, g) \mapsto \gamma \mapsto (f(\gamma), g)$ carries such a pair into $\mathfrak{E}^*_\Gamma$, and Theorem 11 shows this assignment to be a homomorphism. The following commutative diagram states

the same condition: the inverse e returns reverts the transformation at the state where e was applied:



Since effect functions $\mathfrak{E}_\Gamma$ are no longer endomorphisms on the context, they cannot be directly composed. We therefore define a new operation for effect composition:

Definition 9. Given functions $f, g \in \mathfrak{E}_\Gamma$, define their *effect composition* $f \diamond g$ as:

$$\begin{array}{lcll} f \diamond g & : & \Gamma & \to \quad \partial\Gamma \\ f \diamond g & = & \gamma & \mapsto \begin{array}{l} \textbf{let } (\delta, s) = g(\gamma) \textbf{ in} \\ \textbf{let } (\varepsilon, t) = f(\delta) \textbf{ in} \\ (\varepsilon, s \circ t) \end{array} \end{array} \tag{13}$$

Theorem 10. Effect composition carries the monoid structure of $\mathfrak{T}_\Gamma$ over to $\mathfrak{E}_\Gamma$. That is,

1. $(\mathfrak{E}_\Gamma, \diamond)$ is a monoid with unit $\eta_\Gamma := \gamma \mapsto (\gamma, \mathrm{id}_\Gamma)$;
2. the assignment $(f, g) \mapsto \gamma \mapsto (f(\gamma), g)$ is a monoid homomorphism from $\mathfrak{T}_\Gamma$ into $\mathfrak{E}_\Gamma$.

Proof.

1. Associativity and the unit laws follow componentwise from those of $\circ$.
2. Write $e_i = \gamma \mapsto (f_i(\gamma), g_i)$; then $(e_1 \diamond e_2)(\gamma) = (f_1(f_2(\gamma)),\, g_2 \circ g_1)$, which is the image of $(f_1, g_1) \circ (f_2, g_2)$, and $(\mathrm{id}_\Gamma, \mathrm{id}_\Gamma)$ maps to η_Γ. □

Theorem 11. Witnessing survives effect composition, and a uniform inverse witnesses at every state. That is,

1. $\mathfrak{E}^*_\Gamma$ is a submonoid of $\mathfrak{E}_\Gamma$;
2. the homomorphism of Theorem 10 carries every pair with $g \circ f = \mathrm{id}_\Gamma$ into $\mathfrak{E}^*_\Gamma$.

Proof.

1. The unit lies in $\mathfrak{E}^*_\Gamma$ since $\mathrm{id}_\Gamma(\gamma) = \gamma$. For closure, take $f, g \in \mathfrak{E}^*_\Gamma$ and any $\gamma \in \Gamma$, and let $(\delta, s) = g(\gamma), (\varepsilon, t) = f(\delta)$, so that $(f \diamond g)(\gamma) = (\varepsilon, s \circ t)$. Then $s(\delta) = \gamma$ and $t(\varepsilon) = \delta$, therefore $(s \circ t)(\varepsilon) = s(\delta) = \gamma$.
2. $g \circ f = \mathrm{id}_\Gamma$ gives $g(f(\gamma)) = \gamma$ at every γ, so the image of such a pair is witnessed at every state. □

Just as track lifts a pair of transformations on Γ to $\partial\Gamma$, we define effect to lift $\mathfrak{E}_\Gamma$ to $\mathfrak{E}_{\partial\Gamma}$:

Definition 12. Define the effect function transformation effect_Γ as:

$$\begin{array}{lcllll} \mathrm{effect}_\Gamma & : & \mathfrak{E}_\Gamma & \to \quad \partial\Gamma & \to & \partial^2\Gamma \\ \mathrm{effect}_\Gamma & = & e & \mapsto \quad (\gamma, \varphi) & \mapsto & \begin{array}{l} \textbf{let } (\delta, g) = e(\gamma) \textbf{ in} \\ ((\delta, \varphi \circ g),\, \mathrm{track}_\Gamma(g, \mathrm{pr}_1 \circ e)) \end{array} \end{array} \tag{14}$$

Since $\mathrm{effect}_\Gamma(e)$ is itself $\mathfrak{E}_{\partial\Gamma}$, what it returns is an inverse in the sense of Definition 8 read one level up. That inverse is itself a track of the pair obtained by swapping the two directions of the effect. The ordinary tracking rule applies once more: undoing the effect is an effect in its